

Smoothed Perturbation Analysis, From the Beginning

The derivation, the estimator we implement, and what re-declared clocks change

Concourse research notes

July 17, 2026

Contents

1	What this document is	1
2	The setting and the notation	1
2.1	The race	1
2.2	The path as a deterministic map	2
2.3	Where the map is smooth and where it is not	2
3	The smoothing idea	2
3.1	Conditioning makes the discontinuity differentiable	2
3.2	The characterization we use	3
4	The single-segment derivation	3
4.1	The boundary probability and its derivative	3
4.2	Why the jump is estimated with a coupled clone pair	4
4.3	The velocity factor comes from the replay	5
4.4	The validity conditions, collected	5
5	The implemented estimator, in the derivation's notation	5
6	What re-declared clocks change	6
6.1	The vocabulary	6
6.2	What generalizes without new mathematics	6
6.3	Where it actually breaks: the velocity, not the weight	7
6.4	The redraw coupling drops the channel entirely	8
6.5	The two-job worked example	8
7	The experiments	8
7.1	M/M/1-PS, occupancy integral	9
7.2	The degenerate-segment experiment	9
7.3	The refuted one-line repair	9
8	The falsification ladder	9
9	Decisions and open questions	10

1 What this document is

The design conversation asked for a derivation. SPA works in Concourse today for models whose clocks keep one distribution for life, and it refuses records that contain re-declared clocks, because the mathematics for that case had not been worked out. This document is the first step of working it out, following the process the project uses for such questions: write the original derivation carefully, with notation defined and with the reason for each step stated, and only then ask what changes when clocks are re-declared. Books and articles on SPA are terse. This report is deliberately not.

The document also folds in what the Question 3 experiments discovered while it was being written. Those experiments were supposed to be a side check on the pathwise estimator. Instead they located a structural gap that sits underneath both the pathwise estimator and SPA, and the multi-segment part of this report is organized around that finding.

Sections 2 through 4 build the single-segment theory from nothing. Section 5 states the estimator exactly as implemented, so the theory and the code can be compared line by line. Sections 6 and 7 treat re-declared clocks: what generalizes cleanly, what breaks, a small closed-form result that explains the experiments, and the ladder of falsification tests that should gate any implementation. Section 8 collects the decisions.

2 The setting and the notation

2.1 The race

A generalized semi-Markov process runs as a race among clocks. The state is s . In state s a set of clocks is enabled. Clock i was enabled at time τ_i (its *enabling time*) and carries a declared probability distribution F_i for its holding time — the time from τ_i until it would fire. The clock that fires first wins. Firing moves the state, which enables new clocks and disables others, and the race resumes. The model's entire statement about a clock is the pair (F_i, τ_i) , read as the conditional law of its firing time given the history. Nothing else about the randomness is the model's business.

Parameters $\theta \in \mathbb{R}^D$ enter in exactly one place: the declared distributions may depend on θ , so we write $F_i(\cdot; \theta)$. The performance measure is a functional L of the path — a terminal reading, a time integral such as $\int_0^T N(t) dt$, or a first-passage time. We want $\partial_\theta \mathbb{E}[L]$.

2.2 The path as a deterministic map

Give clock i a uniform random number u_i and set its holding time by inversion,

$$X_i = F_i^{-1}(u_i; \theta),$$

so that X_i has law F_i . (When a clock is disabled and later re-enabled, or fires repeatedly, it consumes a fresh uniform each time; index them $u_{i,1}, u_{i,2}, \dots$. Nothing below depends on that detail.) Once every uniform is fixed, the whole path is a deterministic function of θ : each firing time, each state visit, and hence $L = L(\theta, u)$ where u is the vector of all uniforms. The quantity we want is

$$\frac{d}{d\theta} \mathbb{E}[L] = \frac{d}{d\theta} \int L(\theta, u) du.$$

Everything in perturbation analysis is a fight about whether the derivative may be moved inside this integral.

2.3 Where the map is smooth and where it is not

Fix u and move θ a little. Each holding time $X_i = F_i^{-1}(u_i; \theta)$ slides smoothly, because the inverse survival function of a fixed uniform is a smooth function of the distribution's parameters. Firing times are sums and minima of holding times, so they slide too. As long as the *order* of firings does not change, the path deforms continuously: the same events happen, at slightly different times. On this region $L(\cdot, u)$ is typically differentiable, and its derivative is what the pathwise (IPA) estimator computes.

The map is not smooth everywhere. At certain values of θ two firing times tie. Crossing such a value swaps the order of two events, and the path after the swap can be a different path entirely: a different job got the server, a different queue overflowed. If L reads anything that depends on the order — a count, an indicator, a first passage — then $L(\cdot, u)$ jumps at the tie. The set of u for which a tie happens at a given θ has probability zero, which is why the pathwise estimator is well defined; but the set of u for which a tie happens *somewhere near* θ has probability of order $\delta\theta$, and each such u contributes a jump of order one. The product is a term of order $\delta\theta$ that pure pathwise differentiation cannot see:

$$\frac{d}{d\theta} \mathbb{E}[L] = \underbrace{\mathbb{E}\left[\frac{\partial L}{\partial \theta}\right]}_{\text{IPA: the sliding of times}} + \underbrace{(\text{probability flux across ties}) \times (\text{jump size})}_{\text{the boundary term IPA drops}}.$$

The first term is exact when L is continuous at order swaps (a time integral of the state usually is). The second term is the whole story for order-carried observables, and estimating it is what SPA is for.

3 The smoothing idea

3.1 Conditioning makes the discontinuity differentiable

The trick, due to Gong and Ho and developed at book length by Fu and Hu, is not to differentiate L itself but a conditional expectation of L . Choose a *characterization* Z : a set of random quantities that pins down the path except for one distinguished piece of randomness. Then

$$\mathbb{E}[L] = \mathbb{E}[\mathbb{E}[L \mid Z]],$$

and the inner quantity $\mathbb{E}[L \mid Z]$ is a function of θ that integrates over the distinguished randomness. Integration smooths. Where L has a jump indicator $\mathbf{1}\{X_e < \xi\}$ in it, the conditional expectation has the continuous function $\mathbb{P}(X_e < \xi \mid Z)$ instead. If the conditional expectation is differentiable in θ and the interchange is justified, then

$$\frac{d}{d\theta} \mathbb{E}[L] = \mathbb{E}\left[\frac{d}{d\theta} \mathbb{E}[L \mid Z]\right],$$

and the inner derivative now contains the boundary contribution explicitly, as the derivative of a probability. The estimator samples the inner derivative. That is the entire idea. The art is in choosing Z so that the inner derivative is computable from one simulated path plus a small amount of extra work.

3.2 The characterization we use

Concourse’s SPA (following Fu and Hu’s GSMP treatment) applies the idea locally, at each firing of the simulation. Fix an epoch: the winner e_k fires at time t_k . Consider one other clock e that was enabled at that moment — a *candidate* — with enabling time τ_e and age

$$\xi = t_k - \tau_e$$

at the decision. In the realized path the candidate lost: $X_e > \xi$. In a nearby path it might have won, and that is the order swap whose boundary contribution we must capture.

Condition on everything except the candidate’s holding time X_e : all other uniforms, hence all other holding times, hence (up to this epoch) the winner’s firing time t_k and the candidate’s enabling time τ_e . Under this conditioning, what do we know about X_e ? We know the candidate survived every earlier epoch — at each previous firing it was enabled and did not fire. All those survivals are implied by the single statement $X_e > \xi$, because ξ is its age at the latest decision. So the conditional law of X_e given the characterization is its declared law conditioned on survival past ξ :

$$\mathbb{P}(X_e \in dx \mid Z) = \frac{f_e(x; \theta)}{1 - F_e(\xi; \theta)} dx, \quad x > \xi.$$

This is why the boundary weight will be a *hazard* and not a marginal density. The candidate is not a fresh clock; it is a clock we have already watched survive to age ξ , and every probability statement about it must respect that. The implementation pins this with an exact two-clock-race check: with the hazard weight the boundary term integrates to the true derivative $\lambda_b/(\lambda_a + \lambda_b)^2$ of a win probability; with the marginal density it integrates to the wrong value $\lambda_b/(2\lambda_a + \lambda_b)^2$.

4 The single-segment derivation

4.1 The boundary probability and its derivative

Work at one epoch with one candidate, everything else conditioned. Write the conditional expectation of the functional as an integral over the candidate’s holding time x :

$$\mathbb{E}[L \mid Z] = \int_{\xi(\theta)}^{\infty} L(x; \theta) \frac{f_e(x; \theta)}{1 - F_e(\xi(\theta); \theta)} dx.$$

Three things here depend on θ , and keeping them separate is the whole derivation. First, the integrand path $L(x; \theta)$: for x away from ξ the order is fixed and L deforms smoothly — this produces the IPA part, and we set it aside. Second, the density $f_e(\cdot; \theta)$ and its normalization: these produce terms that pair up with the IPA part’s likelihood bookkeeping and, in the implemented estimator, are carried by the same replay. Third — the term we came for — the lower limit $\xi(\theta)$, the boundary between “candidate loses” and “candidate wins.”

Differentiating the integral at its moving lower limit gives the boundary contribution

$$- \frac{f_e(\xi; \theta)}{1 - F_e(\xi; \theta)} \frac{d\xi}{d\theta} L(\xi^+; \theta) + (\text{the matching term in which the boundary point } x = \xi \text{ moves with the law}).$$

The second piece needs one more sentence of care. The boundary is a tie between two *times*, not a fixed number: the candidate wins when its holding time, viewed at fixed uniform as $X_e(\theta)$,

drops below the age $\xi(\theta)$. Both sides move with θ . The rate at which probability crosses the boundary is proportional to the *relative* velocity

$$v = \frac{d\xi}{d\theta} - \frac{dX_e}{d\theta}\Big|_{X_e=\xi}, \quad \frac{dX_e}{d\theta}\Big|_{X_e=\xi} = -\frac{\partial_\theta F_e(\xi; \theta)}{f_e(\xi; \theta)},$$

where the last identity is the fixed-uniform (probability-integral transform) derivative of $X_e = F_e^{-1}(u; \theta)$ evaluated at the boundary point. When $v > 0$ the age is catching up to the candidate’s sample: order swaps flow *into* the swapped configuration at rate $h_e(\xi) v$, where

$$h_e(\xi) = \frac{f_e(\xi; \theta)}{1 - F_e(\xi; \theta)}$$

is the hazard. When $v \leq 0$ the boundary recedes and no probability crosses; this is why the implemented factor is $(v)^+$, the positive part.

What multiplies the flux is the *jump*: the difference between the functional’s value on the path where the pair fired in swapped order and its value on the nominal path, in the limit where the holding time between the two firings vanishes. Call these the perturbed path (PP: candidate first) and the degenerated nominal path (DNP: winner first, candidate immediately after, if still enabled). The local boundary contribution at this epoch and candidate is

$$h_e(\xi) \left(\frac{d\xi}{d\theta} - \frac{dX_e}{d\theta}\Big|_{\xi} \right)^+ (L(\text{PP}) - L(\text{DNP}))$$

and the SPA estimator is the IPA term plus the sum of these contributions over epochs and candidates, plus one more family of terms at the horizon (next subsection). Summing over epochs is legitimate because each epoch’s contribution came from conditioning on a different characterization; Fu and Hu’s Theorem 3.3 hypotheses (differentiable conditional expectation, uniformly integrable difference quotients) are what license both the interchange and the sum.

4.2 Why the jump is estimated with a coupled clone pair

$L(\text{PP})$ and $L(\text{DNP})$ are values of the functional on two constructed continuations of the same prefix. The construction must respect the characterization: everything except the candidate’s residual is held fixed, so both continuations start from the same state, fire the same two events at the same instant t_k (in opposite orders), and then continue *with the same future randomness*. In the implementation both clones branch from the pre-commit world, receive one shared rekey seed, force the pair in their respective orders, and run to the horizon. Sharing the seed makes the difference $L(\text{PP}) - L(\text{DNP})$ quiet: the continuation randomness cancels and only the effect of the order remains. An independent-seeds version would be unbiased too, but its variance would swamp the term being estimated.

Two structural facts reduce the cost. First, the *commuting gate*: if firing the pair in both orders re-coalesces to the same state, with both events surviving in both orders, then under the characterization the two continuations are the same path and the jump is exactly zero — no clones needed. This is a correctness statement, not just a saving: the clone pair in that case would return pure noise. (A first-passage functional needs a stronger gate, because it reads the zero-duration intermediate states.) Second, the *horizon term*: a clock scheduled just past the observation time T crosses into the window as θ moves. That is a boundary with the constant T , so its “decision-time derivative” is zero, no clones are needed, and the jump is read directly off the fired state. A time integral has zero jump there (the crossing event occupies zero width), consistent with IPA already being exact for it.

4.3 The velocity factor comes from the replay

The term $d\xi/d\theta = dt_k/d\theta - d\tau_e/d\theta$ requires the derivative of two realized *times* with the order frozen — exactly what the pathwise replay computes. The implementation takes the Jacobian of `replay_times` at the recorded path and reads off the rows for the winner’s epoch and the candidate’s enabling epoch. The term $dX_e/d\theta|_\xi$ comes from the candidate’s law directly ($-\partial_\theta F/f$ at the frozen age ξ), never from the replay: the candidate did not fire in the realized path, so it has no replayed time, and using a replayed dual age inside $\partial_\theta F$ would count θ twice.

This dependence is worth stating loudly, because Section 6 turns on it: *the boundary term is built on top of the pathwise replay*. If the replay’s time derivatives are wrong, the SPA weight multiplies wrong velocities, and no boundary correction can rescue the sum.

4.4 The validity conditions, collected

For the single-segment estimator as implemented, the requirements are:

1. Each clock keeps one declared law from enabling to firing, so the conditional law given survival is read off that one law (the hazard at the age).
2. The clock laws are dual-safe: the replay’s inverse survival functions differentiate under ForwardDiff.
3. `fire` is pure and states compare by value, so the commuting gate can fire speculatively.
4. After forcing the pair, each surviving clock’s continuation law is its declared law conditioned on its own age — the world’s rekey-then-jitter performs exactly this resample-given-survival at a stopping time.
5. The functional and laws satisfy the interchange hypotheses (differentiable conditional expectations, uniformly integrable quotients). We do not verify these per model; the finite-difference comparisons in the charter are the operational check.

5 The implemented estimator, in the derivation’s notation

For reference, the map between the theory and `ClockGradients/src/spa.jl`:

Theory	Implementation
IPA term	<code>ipa_gradient</code> on the replication’s own record
hazard $h_e(\xi)$	<code>pdf(d, ~xi)/ccdf(d, ~xi)</code> at the enabling-state law
$d\xi/d\theta$	rows of <code>ForwardDiff.jacobian(replay_times)</code>
$dX_e/d\theta _\xi$	<code>-ForwardDiff.gradient(ccdf)/f(xi)</code>
positive part $(v)^+$	<code>max(dxi - dx, 0.0)</code>
jump $L(\text{PP}) - L(\text{DNP})$	coupled clone pair, shared rekey seed
commuting gate	<code>zero_jump_certified</code> (two speculative fires)
horizon family	<code>_horizon_jump</code> , no clones
candidate enumeration	all enabled non-winners (<code>HazardWeight</code>), or next winner with truncated hazard (<code>TruncatedHazard</code>)
multi-segment refusal	<code>has_chains(record)</code> throws

The refusal in the last row is the subject of the rest of this document. Per the design conversation, it stays a whole-record refusal: if any part of a record is beyond the estimator’s derivation, the user hears a clear no rather than a quietly wrong number.

6 What re-declared clocks change

6.1 The vocabulary

Under processor sharing (and any future discipline that re-evaluates laws mid-flight), a clock’s declared distribution is replaced at event times while the clock stays enabled. Each replacement starts a new *segment*. The enabling time τ_e stays fixed; what changes is the declared law, which after a re-declaration says: given the work this job has received (the banked age a) and the current sharing speed, here is the conditional law of the firing age. In Concourse the declared object is `SharedRemaining(base, bank, speed, shift)`, whose support begins at the segment’s start.

Write the segment boundaries of one clock as ages $0 = a_0 < a_1 < \dots < a_m$ past τ_e (each a_i is some other firing’s time minus τ_e), with declared laws $G_1, \dots, G_{m+1}$ on the successive segments. The boundaries are event times of the same race we are differentiating, so each a_i depends on θ .

6.2 What generalizes without new mathematics

The weight. The characterization argument in Section 3.2 never used the number of segments. Conditioning on everything except the candidate’s residual randomness, the candidate’s conditional law at the decision is its *current* declared law given survival to its current age. So the boundary weight for a re-declared candidate is the hazard of the latest segment’s declared law at the age since τ_e :

$$h_e(\xi) = \frac{g_{m+1}(\xi)}{1 - G_{m+1}(\xi)},$$

with G_{m+1} built at the state of the last re-declaration rather than the state of the enabling. In the implementation this means the candidate bookkeeping must track the last re-declaration state, a mechanical change.

The clone construction. By the declaration rule, the re-declared laws after a forced firing depend only on the post-firing state and the clocks’ own histories. If the two orders of the forced pair re-coalesce to the same state — and for a processor-sharing station two completions in either order leave the same job set — then the survivors’ re-declared laws coincide in both arms, and the commuting gate’s conclusion (zero jump) still holds. Forcing *always* re-declares every survivor under PS, but it re-declares them identically in the two arms whenever the states match. The gate survives.

6.3 Where it actually breaks: the velocity, not the weight

Section 4.3 said the boundary term is built on the pathwise replay. The Question 3 experiments (Section 7) show that the pathwise replay is biased on multi-segment records — under both couplings, in different directions, even when every segment declares the same conditional law. So the multi-segment problem is not, first, an SPA problem. It is a problem in the object SPA stands on.

The cleanest statement of what is missing comes from differentiating the carry chain itself. The chain maps a would-be firing age across a re-declaration by matching conditional survival: if

under the old law G the clock would fire at age α , and at boundary age a the law is replaced by G^{new} , the new would-be age α' solves

$$\log S^{\text{new}}(\alpha') = \log S^{\text{new}}(a) + \log S(\alpha) - \log S(a), \quad S = 1 - G.$$

This is exactly the transformation the sampler applies at a re-enable, and it is what `_replay_carry` evaluates. Now differentiate the identity with respect to anything — a component of θ , with the boundary age $a = a(\theta)$ moving too. Using $\partial_x \log S = -h$ (the hazard) and writing dots for derivatives:

$$\dot{\alpha}' = \frac{h^{\text{new}}(a) - h(a)}{h^{\text{new}}(\alpha')} \dot{a} + \frac{h(\alpha)}{h^{\text{new}}(\alpha')} \dot{\alpha} + \frac{\partial_\theta \log S(\alpha) - \partial_\theta \log S(a) + \partial_\theta \log S^{\text{new}}(a) - \partial_\theta \log S^{\text{new}}(\alpha')}{h^{\text{new}}(\alpha')}$$

Read the first coefficient. The sensitivity of the *boundary time* enters the clock's replayed future weighted by the *hazard jump* across the re-declaration, $h^{\text{new}}(a) - h(a)$. Three consequences, and each one matches an experiment:

1. If a re-declaration does not change the conditional law ($h^{\text{new}} \equiv h$ across the boundary), the coefficient is zero and the chain step is transparent: a degenerate multi-segment clock must replay exactly like a single-segment one.
2. If the re-declaration changes the speed (processor sharing), the coefficient is proportional to the speed change, and the boundary time's derivative $\dot{a}$ — the replayed derivative of some other firing — genuinely feeds the clock's future. Dropping it biases the replay.
3. The current implementation cannot express the first term. The segment laws are rebuilt from states folded at the *recorded* Float64 times, so `bank`, `anchor`, and speed are frozen constants with no derivative; and the evaluation of $\log S^{\text{new}}$ at the boundary lands exactly at the new law's support start, where the value is pinned to zero regardless of how the boundary moves. Half of the channel ($h(a)\dot{a}$, through the old law) flows; the other half ($h^{\text{new}}(a)\dot{a}$, through the new law) is killed. The two halves are supposed to cancel exactly in the degenerate case — killing one of them is why the degenerate experiment shows large per-path errors, not small ones.

A one-line repair was tried and refuted: taking the formula branch at the support boundary changes nothing, because round-off places the evaluation on either side of the boundary and the frozen anchors remain frozen either way. The repair that the algebra asks for is structural: the segment parameters inside the replayed laws must be dual-valued functions of the replayed times. Concretely, either the replay folds states at dual times (so `bank` and `anchor` carry derivatives), or the record stores each firing's segment boundary *indices* (which firing re-declared this clock) so the replay can rebuild boundary ages as differences of replayed times — a representation question, and precisely Question 4 of the design brief, now with a derivative-driven reason to answer it.

6.4 The redraw coupling drops the channel entirely

The same algebra explains the redraw coupling's numbers. Its recurrence anchors each firing at the *last* re-declaration: $t = \tau_e + \Phi^{-1}(\log u + \Phi(t_{\text{draw}} - \tau_e))$ with $\Phi = \log S$ of the last segment's law. The conditioning term $\Phi(t_{\text{draw}} - \tau_e)$ evaluates at the last segment's own start, where it is pinned to zero, so the derivative of t_{draw} never enters: each firing's sensitivity reduces to the last

segment’s residual only. On the M/M/1–PS experiment this truncation shows up as $\partial_\mu \approx -10$ against a true -47.7 : the estimator sees only the tail of each service, not the θ -dependence of the whole shared history.

6.5 The two-job worked example

The design brief asked for a two-job hand example, and it is worth recording because it shows where processor sharing hides its jumps.

Two jobs share one server, no further arrivals, service requirements S_1, S_2 drawn from $F(\cdot; \theta)$. While both are present each is served at speed $\frac{1}{2}$; alone, at speed 1. The first completion is at $2 \min(S_1, S_2)$; the second at $\min(S_1, S_2) + \max(S_1, S_2)$ — work conservation: the total busy period is $S_1 + S_2$ regardless of order. The occupancy integral is

$$\int_0^\infty N(t) dt = 2 \cdot 2 \min + (\min + \max - 2 \min) = 3 \min(S_1, S_2) + \max(S_1, S_2),$$

a continuous, symmetric, piecewise-smooth function of (S_1, S_2) . At the near-tie $S_1 = S_2$ the swap exchanges two identical futures: the jump $L(\text{PP}) - L(\text{DNP})$ is zero, and the commuting gate would prove it (two completions in either order leave the empty station). So for occupancy-type functionals of a plain PS station, the boundary term contributes nothing at completion–completion ties, and the entire derivative must come through the pathwise term — through $\min$ and $\max$ sliding with θ , including the way each completion’s time depends on the *other* job’s completion through the speed change. This is exactly the channel Section 6.3 shows the replay to be missing. The lesson: for processor sharing the hard part of SPA is not the exotic boundary weight; it is that the smooth part must first be made correct. Jumps that need the weight arise one level up, when the swap changes something downstream that does not re-coalesce — a routing decision, a blocking threshold, a class boundary.

7 The experiments

Three experiments, all with fixed seeds, all against common-random-number central finite differences (FD) on the same seeds; details in `notes/event_loop.tex` and the pinned charter tests.

7.1 M/M/1–PS, occupancy integral

$\lambda = 1$, $\mu = 2$, horizon 50, 400 records. Every record carries multi-segment chains.

	∂_λ	∂_μ
FD	93.4 ± 3.9	-47.7 ± 1.8
IPA, redraw	57.7 ± 3.4	-10.0 ± 0.1
IPA, carry	206.0 ± 11.8	-36.2 ± 1.2

Both couplings biased, in different directions; duals flow with no errors; the Float64 replay reproduces the recorded times. Pinned as charter item F14.

7.2 The degenerate-segment experiment

The design conversation proposed the decisive control: multi-segment records whose segments all declare the same conditional law. A 64-server PS station serves every job at speed 1 at this load ($\lambda = 3$, $\mu = 1$; a slowdown would need 65 simultaneous jobs), so its law is identical to the same station under FCFS — but the PS records carry chains (300 of 300) while the FCFS records are segment-free.

∂_μ (true ≈ -143)	segment-free records	degenerate chains
FD	-143.1 ± 1.0	(same law)
IPA, redraw	-142.8 ± 1.0	-29.5 ± 0.1
IPA, carry	-142.8 ± 1.0	-100.4 ± 0.6

Identical dynamics, opposite verdicts. The bias lives entirely in the record’s segment representation, which is what Section 6.3’s algebra predicts: the degenerate case should be transparent ($h^{\text{new}} \equiv h$), and it fails only because the implementation expresses half of a cancelling pair. Pinned as the sharpened F14 test.

7.3 The refuted one-line repair

Taking the formula branch at the support boundary of **SharedRemaining** (strict instead of weak inequality) changed no number in either experiment: bit-identical gradients. Diagnosis in Section 6.3; kept here because a refuted repair is as informative as a successful one, and because it rules out the cheap fix before anyone tries it again.

8 The falsification ladder

The derivation dictates the order of implementation work, each step gated by a test that can refute it.

1. *Segment-aware replay.* Give the replay the missing channel: rebuild segment laws with dual boundary parameters (fold at dual times, or record segment boundary indices — the Question 4 decision). Gate: the degenerate-segment test flips from refutation to agreement, path by path — carry on degenerate chains must equal the segment-free replay exactly, not just on average.
2. *Pathwise on PS.* Gate: the M/M/1-PS experiment’s carry column matches FD; then a Weibull-base PS variant (non-exponential, dual-safe) matches FD, which exercises the $\partial_\theta \log S$ terms the exponential case partially hides.
3. *SPA weight for re-declared candidates.* Implement the latest-segment hazard of Section 6.2, replacing the whole-record refusal only after step 2 holds. Gate: on a functional where PS boundary terms vanish (occupancy), SPA must agree with the repaired IPA; on an order-carried functional (terminal count through a downstream contended resource), SPA must match FD where plain IPA visibly does not.
4. *Jump construction audit.* The clone pair under PS re-declares survivors in both arms; the commuting gate covers the re-coalescing cases. Gate: an adversarial model where the two orders do *not* re-coalesce (a blocking threshold read between the pair), checked against FD.

Until step 2 passes, both the SPA refusal and a matching refusal for IPA on chained records are the user-facing truth. The design conversation settled that refusals are whole-record: a record either is within the derivation or is not.

9 Decisions and open questions

- The hazard weight generalizes to re-declared candidates as the latest declared law’s hazard at the age since enabling, built at the last re-declaration state. No new mathematics; bookkeeping only.
- The commuting gate survives re-declaration, by the declaration rule, whenever the forced pair re-coalesces.
- The genuinely new mathematics is the chain derivative of Section 6.3. Its hazard-jump coefficient identifies exactly which sensitivity the current replay cannot represent, predicts the degenerate experiment’s failure, and reduces to the classical recurrence when no law changes.
- The implementation decision it forces is representational — Question 4’s “where does segment truth live” — and should be taken with this document in hand: the reconstruction keeps the record minimal (the filtration), so the preferred route is to make the reconstruction dual-capable (fold at dual times), and to revisit storing boundaries only if that proves too costly.
- Open: the uniform-integrability hypotheses for the multi-segment interchange; whether the truncated-hazard strategy’s residual (η) needs a segment-aware form; and θ -dependent marks, which stay excluded until the mark derivative term is derived.